People's Democratic Republic of Algeria
University of Algiers 1 – Benyoucef Benkhedda

From Reactive Supervision to Proactive Prediction:

Multi-Agent Architecture for Anticipating IT Resource Needs

- **Topic proposed by:**
  - Mrs. Benzaid Chafika
  - Mr. Zeraoulia Khaled
- **Supervised by:**
  - Mrs. Ali Saoucha Naziha
- **Jury membres:**
  - Mr. Abbas Amine
  - Mr. Adouane Mohammed Amine
- **Realized by:** Mr. Yassa Rafik and Mr. Bouabache Younes

Master2 : Networks and Embedded Systems

2025/2026

Presentation Roadmap

01 Context & Problem Statement

02 Limitations & Alignment

03 Conceptual Design

04 Implementation & Deployment

05 Experimental evaluation

06 Conclusion

1. Context & Problem Statement

CONTEXT & PROBLEM STATEMENT

1.1 Context and Motivation

Over the past decade, the infrastructure behind digital services took a significant shift.

Rapid Cloud Adoption:
On-premise hardware replaced by cloud resources under container orchestration

Architectural Shift:
Decomposition of traditional monolithic applications into constellations of independently deployable microservices.

The Resource Management Challenge:
Workload heterogeneity and dynamic cloud elasticity make efficient and timely resource management more critical than ever.

THE PROBLEM WITH THIS IS :

The systems that manage and supervise cloud infrastructure have not evolved at the same pace as the systems they govern.

CONTEXT & PROBLEM STATEMENT

1.2 Problem Statement

REACTIVE APPROACH

Responds after degradation

- Dominant in production for its simplicity and ease of deployment
- Thresholds configured → violations detected → action triggered after degradation
- Not efficient to handle the shift that happened, it acts only once a threshold is breached

PROACTIVE APPROACH

Anticipates before saturation

- Exists and is well studied, but not universally adopted
- Predicts future resource states and acts within a horizon preceding saturation
- Can be the answer to the shift but contingent on forecast accuracy and limited by centralized architectures

Centralization introduces:

Centralized control - Serialized decision-making - Single point of failure - Global, stale, offline-trained models

Effect:

Effective intervention still depends on human expertise applied after the fact.

CONTEXT & PROBLEM STATEMENT

1.3 Research Question

PRIMARY

Can a multi-agent architecture where autonomous agents perform localized forecasting and scaling decisions, provide a more scalable and responsive resource management layer than both reactive and centralized supervision in a cloud-native environment?

2. Limitations & Alignment

LIMITATIONS & ALIGNMENT

2.1 Reactive Approach Limitations

The dominant production approach configures thresholds, detects violations, and triggers corrections only after degradation has already materialized, which makes the anticipation window the critical gap.

Inherent Activation Lag

The system must wait until a threshold is breached before any action is taken — corrections arrive after degradation has materialized.

Provisioning Delay Blindspot

Resource provisioning is not instantaneous. Under bursty workloads, new capacity arrives too late to prevent SLA violations.

Burst-Oblivious Behavior

Most reactive autoscalers fail to detect and mitigate burstiness, leading to response-time SLO violations and oscillatory scaling decisions.

Threshold Calibration Difficulty

Choosing the right metrics and thresholds is hard. Effectiveness is tightly tied to workload behavior and breaks under irregular patterns.

LIMITATIONS & ALIGNMENT

2.2 Limitations of Centralized Supervision

Sequential pipeline latency

Each step — collect, aggregate, analyze, plan, execute — adds delay embedded in the chain itself.

Model rigidity & Concept drift

Global models on aggregated data miss per-service patterns; retraining lags workload evolution.

Fault intolerance

Centralized controllers form single points of failure, most likely to fail under peak load.

Metric overload & Cognitive load

Large telemetry volumes overwhelm static threshold mechanisms and human operators.

LIMITATIONS & ALIGNMENT

2.3 MAS Properties Alignment:

The structural properties of a decentralized intelligent MAS map directly onto the two limitations of centralization and reactivity.

STRUCTURAL CORRESPONDENCE

Prediction and Forecast-driven anticipation	*addresses* →	Reactive threshold triggering
Local autonomy	*addresses* →	Sequential latency
Distributed fault isolation	*addresses* →	Fault intolerance
Lead time ≥ provision delay	*addresses* →	- Provisioning-delay exposure - SLA/SLO violations
Decentralized state	*addresses* →	Model rigidity
Localized telemetry processing	*addresses* →	Metric & cognitive overload

3. Conceptual Design

CONCEPTUAL DESIGN

3.1 Assumptions:

- Secure operating environment
- Reliable network conditions
- Optimal parameter configuration

CONCEPTUAL DESIGN

3.2 Global Architecture

Multi-Agent System

Intelligence layer: observation, forecasting, scaling, recovery and oversight.

Communication Framework

Asynchronous messaging that keeps agents loosely coupled and non-blocking.

Knowledge Base

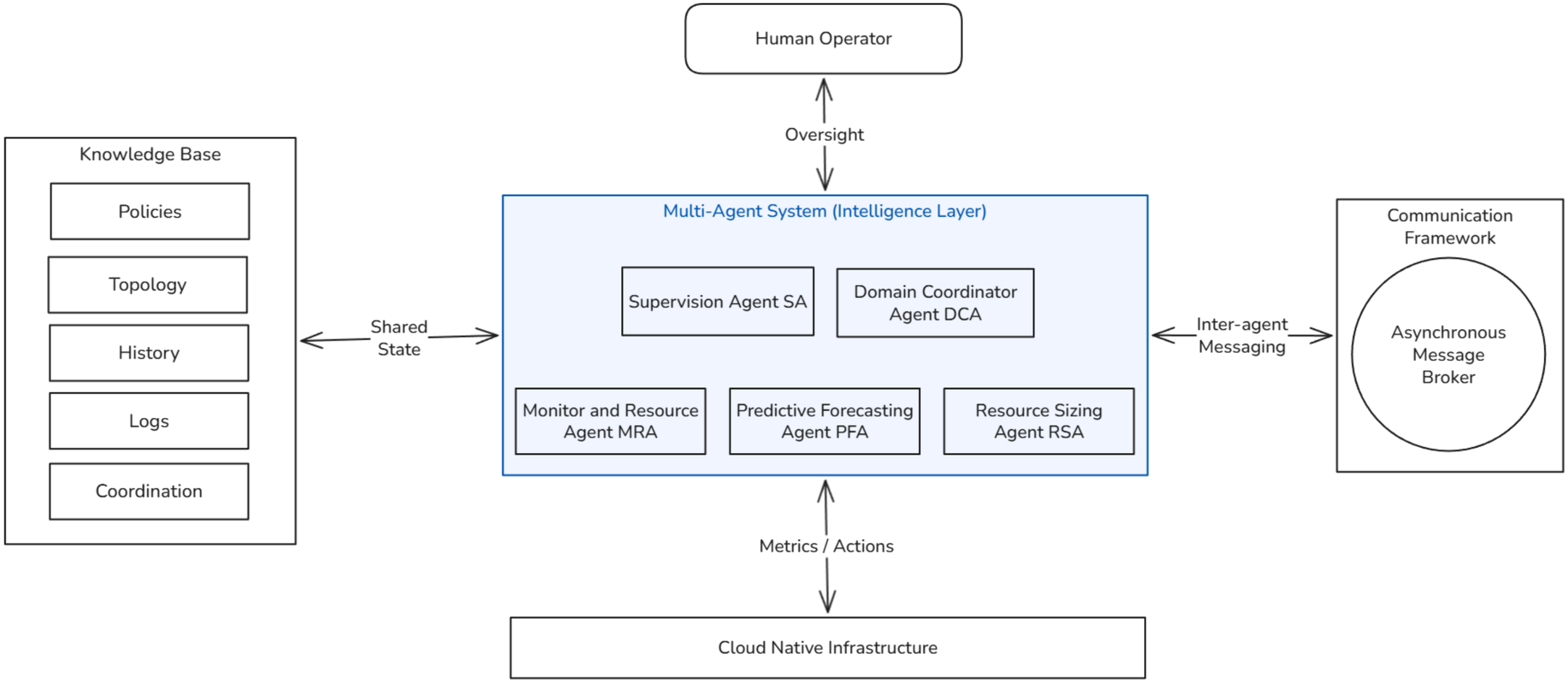
Shared, durable state: policies, topology, metric history, audits, and coordination.

Cloud-native infrastructure

The managed and monitored target, it is the metric source and scaling execution surface.

CONCEPTUAL DESIGN

3.2 Global Architecture



CONCEPTUAL DESIGN

3.3 MAS Architecture : Two-Tier Hierarchy

TIER 2 · OPERATIONAL PIPELINE

Monitor and Resource Agent (MRA)
Monitor

Predictive Forecasting Agent (PFA)
Analyze / Forecast

Resource Sizing Agent (RSA)
Plan + Execute

Structural independence:
the operational tier executes its cycle without relying on Tier 1 — preserving continuity under partial coordination failure.

CONCEPTUAL DESIGN

3.3 MAS Architecture : Agent Distribution

The agent placement strategy aligns each agent with the scope where it provides the greatest value while ensuring service isolation, fault containment, and scalability.

Service-Level Agents (One instance per managed service):

MRA: Dedicated monitoring and isolated telemetry collection.

PFA: Service-specific demand forecasting for higher prediction accuracy.

RSA: Independent resource sizing and provisioning with minimal scaling latency.

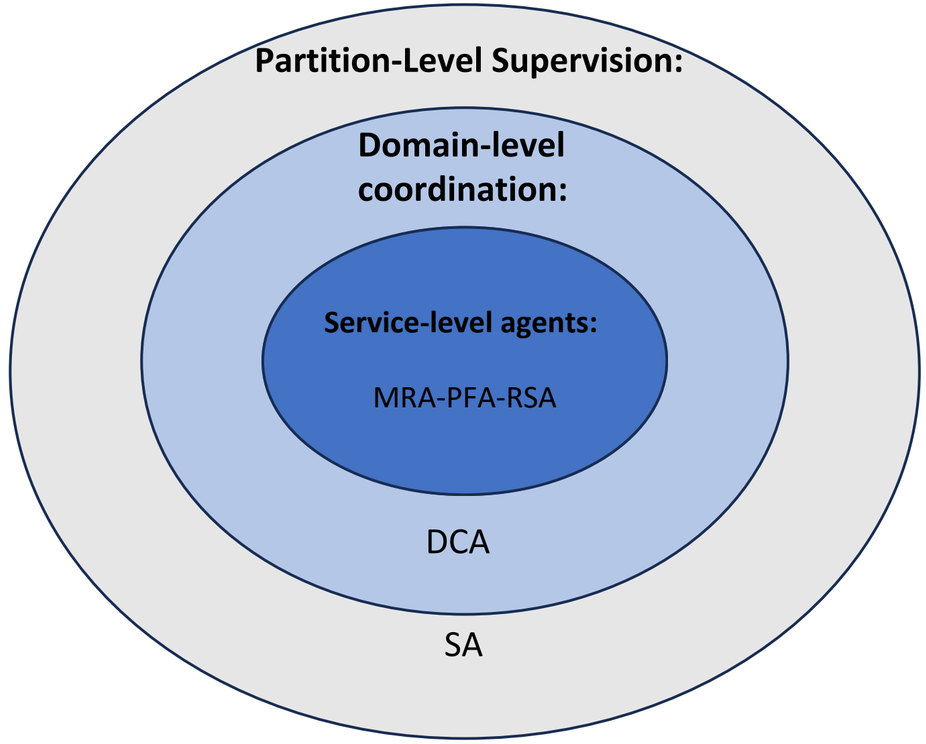
Together, these agents form an autonomous resource management pipeline for each service.

Domain-Level Coordination:

DCA: One instance per administrative domain to coordinate multiple triads, prevent bottlenecks, and localize failures.

Partition-Level Supervision:

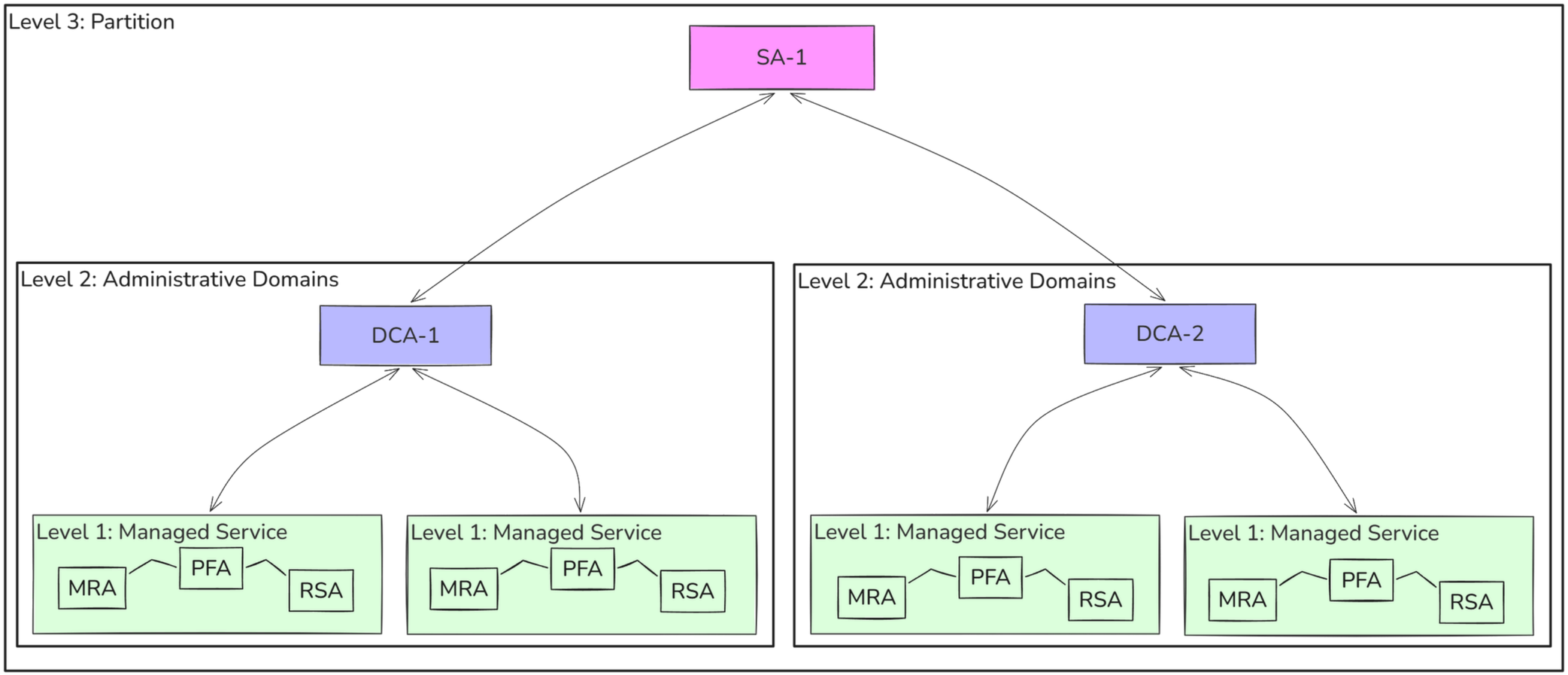
SA: One instance per DCA group (partition) to provide balanced system-wide oversight while limiting the impact of supervisory failures.



CONCEPTUAL DESIGN

3.3 MAS Architecture : Topology

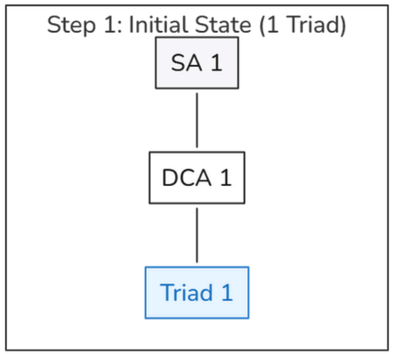
The defined distribution strategy and architecture result in a topology with three levels. Each level carries a well-defined responsibility.

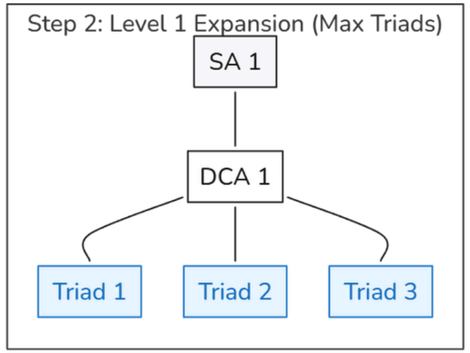


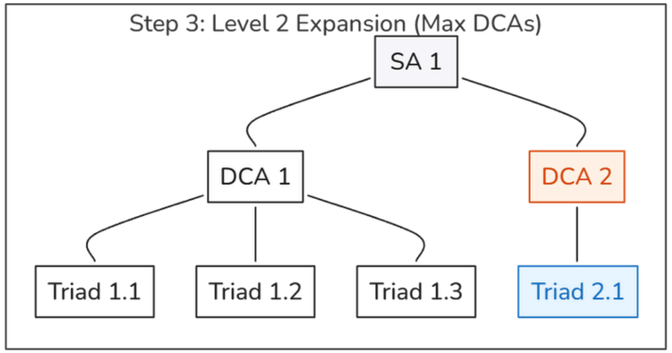
CONCEPTUAL DESIGN

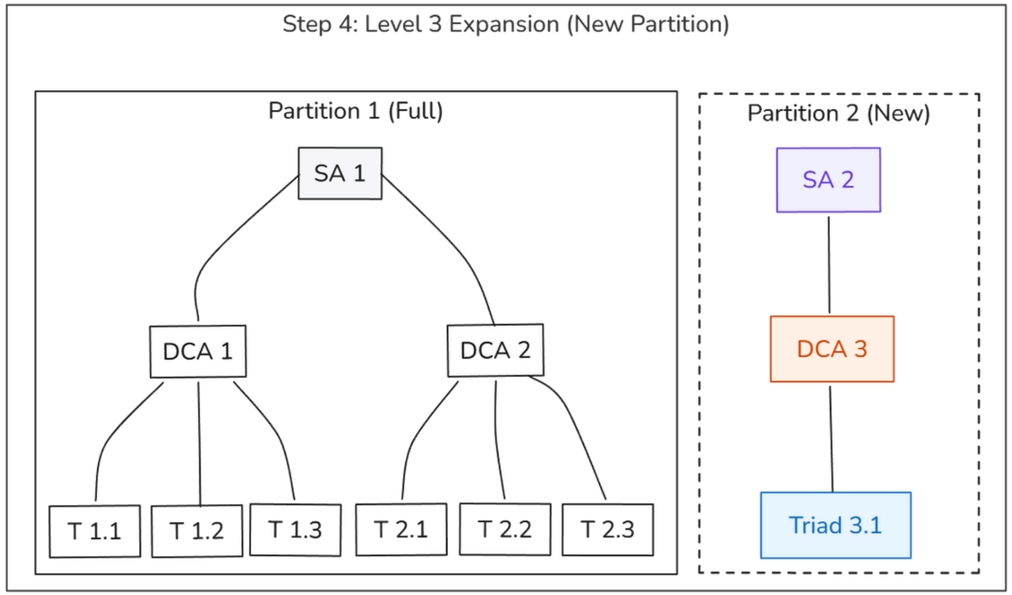
3.3 MAS Architecture : Topology

The topology grows by alternating expansion across levels by adding triads, then DCAs, then SAs , until hardware capacity is reached.









CONCEPTUAL DESIGN

3.3 MAS Architecture: Human-in-the-Loop & Policies

The operator retains ultimate authority while agents handle all routine decisions autonomously — escalating only when conditions exceed their authorization scope.

HUMAN OVERSIGHT MODEL

- **Policy Definition**
- **Action on Escalations**
- **Forced Direct Action**
- **Emergency Stop**

POLICY

DEFINE | VALIDATE | PROPAGATE | ENFORCE

- **SLA/SLO-Aligned**
- **SA-Validated**
- **Immutable Before Propagation**

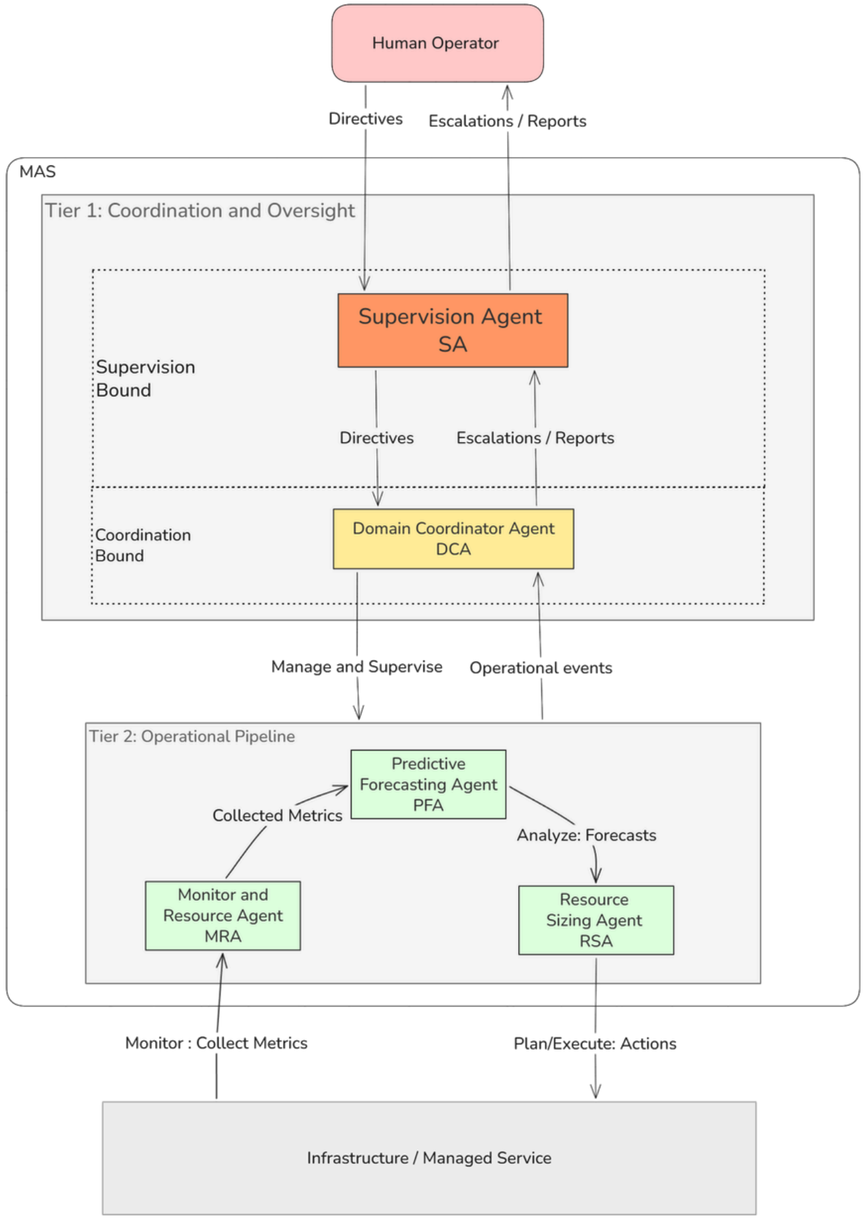
CONCEPTUAL DESIGN

3.3 MAS Architecture: Agent Roles and BDI Modeling

AGENT	ROLE	BDI VARIANT
MRA	Telemetry collection, Normalization, Validation, Pressure classification	BDI fixed pipeline
PFA	Forecasts future demand, Computes uncertainty and Time-to-breach	BDI fixed pipeline
RSA	Sole authority for Sizing & Provisioning	Hybrid BDI deliberative + reactive
DCA	Management, Coordination, Recovery, Filters, Escalates, Reports	Pure BDI
SA	Organize, Validate, Present, Suggest actions, Reports	Pure BDI

CONCEPTUAL DESIGN

3.3 MAS Architecture: Overview and Flow



CONCEPTUAL DESIGN

4. Communication and Knowledge Base

COMMUNICATION

Asynchronous publish-subscribe via broker-mediated messaging:
Loose coupling
Non-blocking execution
Fan-out support

Six performatives

INFORM	REPORT	ALERT
NOTIFY	ESCALATE	DIRECTIVE

Publication & Absence interpretation rules make agent behavior well-defined and observable.

KNOWLEDGE BASE · FIVE DOMAINS

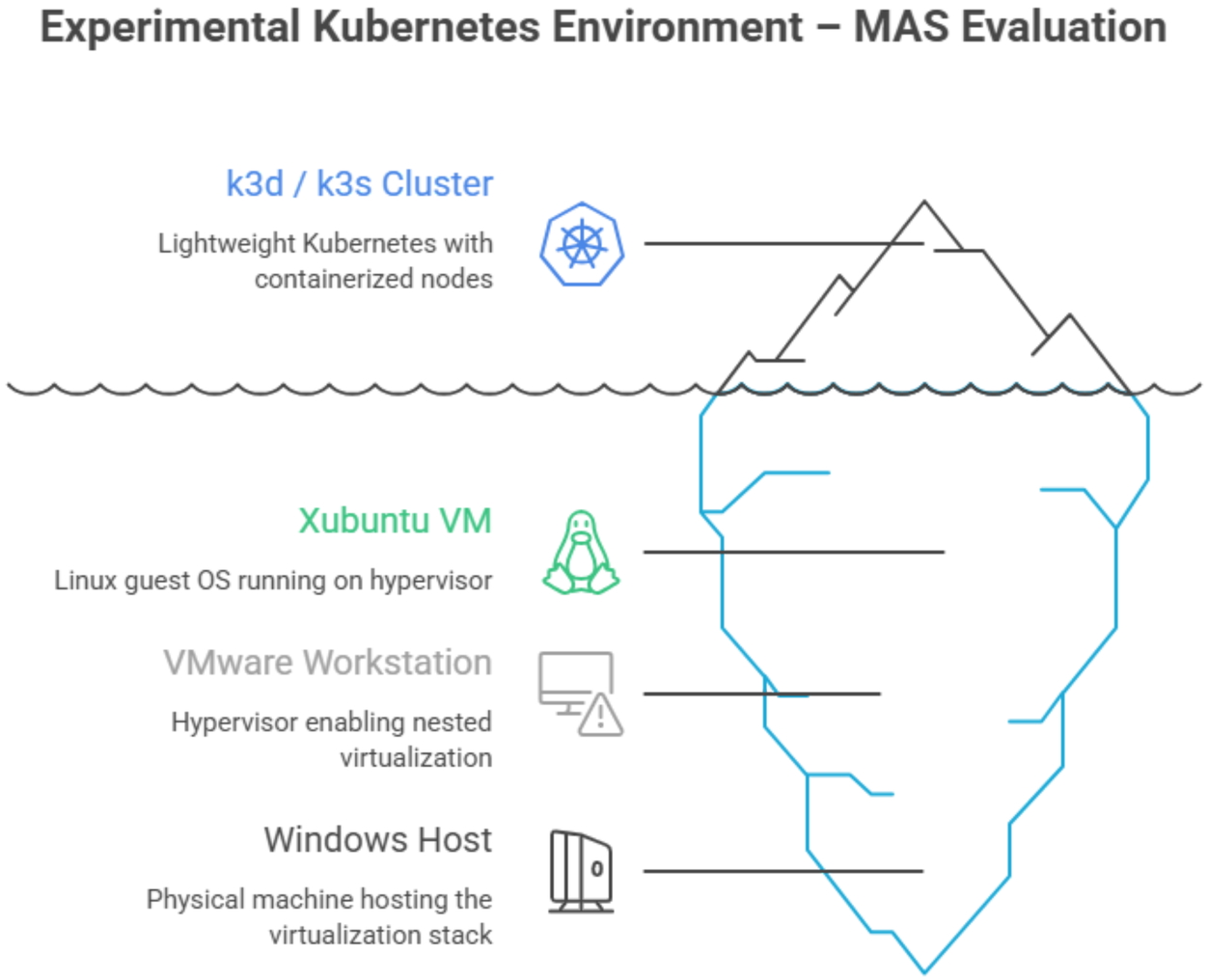
Policy	*SA writes • others: read*
Topology	*operator-maintained, all: read*
Historical State	*MRA writes • PFA reads*
Audit Log	*append-only, all agents write*
Coordination	*DCA reads and writes*

Strict role-based access: one writer per domain eliminates write-write conflicts — no distributed locking.

4. Implementation & Deployment

IMPLEMENTATION & DEPLOYMENT

4.1 Experimental Environment & Implementation Technologies



Python

Agent runtime — K8s client, paho-mqtt, ML ecosystem

Docker / Kubernetes

Multi-stage agent images; namespaces & RBAC; replica scaling via patch ops

SQLite (WAL)

Audit Log & Coordination Store, backed by PersistentVolume

MQTT (Mosquitto)

Lightweight async pub/sub backbone for inter-agent coordination, QoS levels

TimesFM 2.5

Zero-shot forecasting engine — quantile (P10/P50/P90) multi-step forecasts, 15-min horizon @30s resolution

Helm

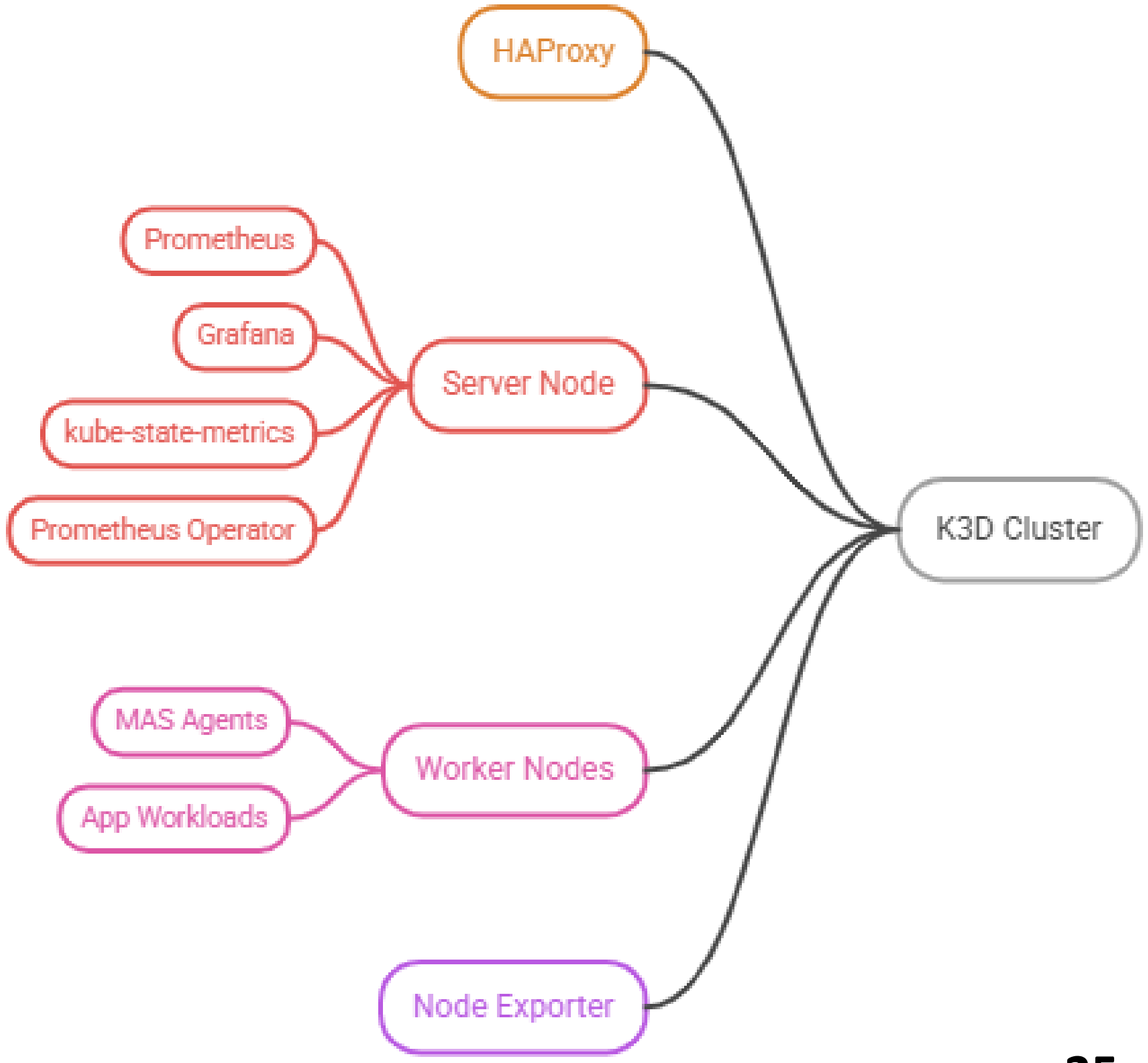
Declarative, reproducible monitoring stack deployment

IMPLEMENTATION & DEPLOYMENT

4.2 Infrastructure Setup

- K3d cluster runs k3s V1.31.5 across 5 containerized nodes
- Control plane isolated on server-0 with a NoSchedule taint, no workloads scheduled here
- Monitoring stack pinned to server-0 (Prometheus, Grafana, Kube-state-metrics, and Prometheus Operator)
- Mas agents and application workloads restricted to worker nodes via nodeAffinity on role=worker
- Node Exporter deployed as a DaemonSet on all nodes for hardware-level metrics collection

This placement strategy separates orchestration and observability infrastructure from the supervised workloads and MAS agents running on worker nodes.

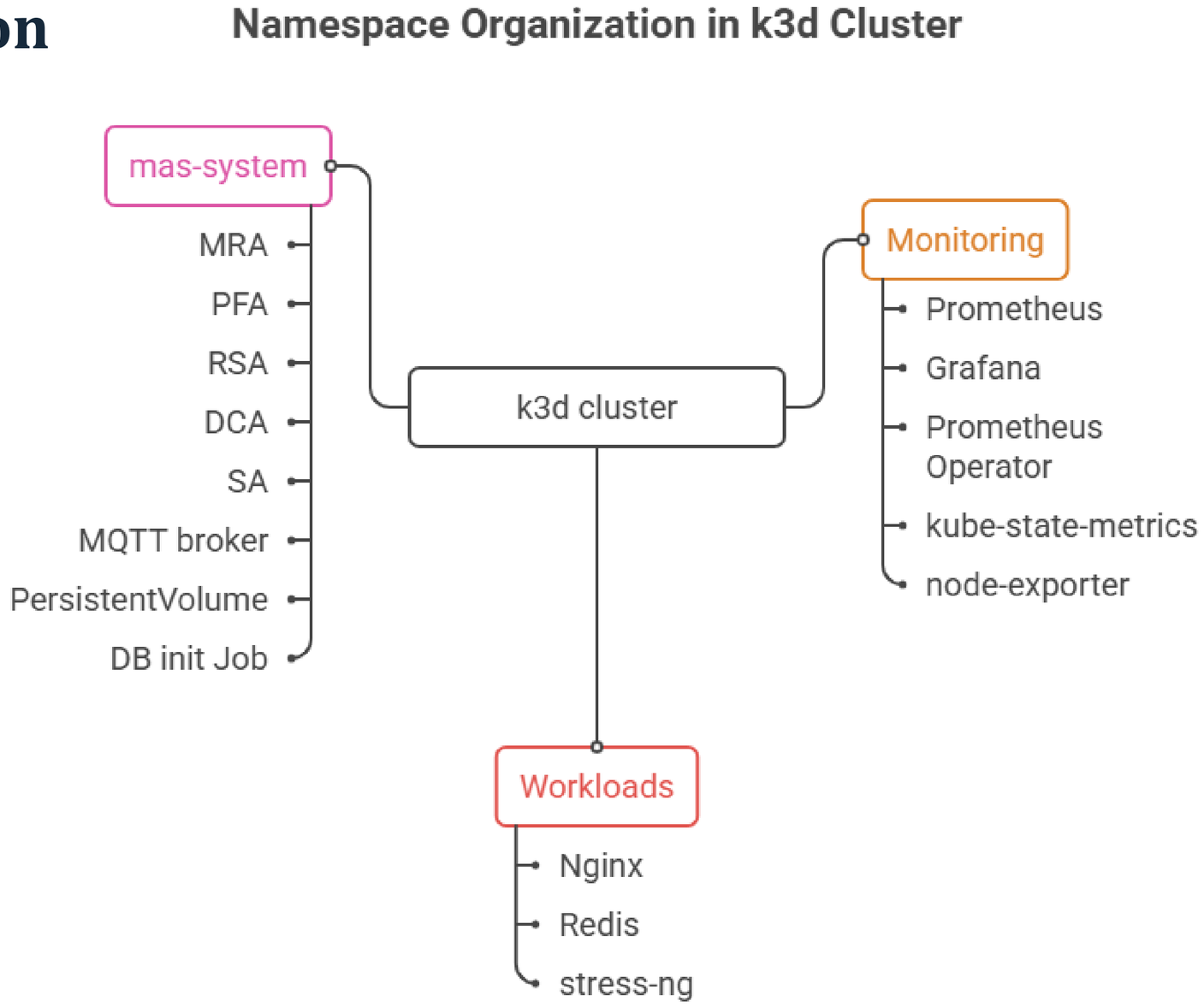


IMPLEMENTATION & DEPLOYMENT

4.3 Namespace Organization

- 3 namespaces partition the cluster into distinct functional layers
- **Mas-system :** hosts all MAS agents, the MQTT broker, shared PersistentVolume, and the DB init; RBAC scoped to this namespace
- **Monitoring:** full kube-Prometheus-stack deployment; isolated from supervised workloads to avoid interference with observations
- **Workloads**: supervised services; resource utilization patterns observed by MRA and modified by RSA scaling actions

Namespace boundaries enforce isolation between what is supervised, what supervises, and what observes keeping experimental concerns cleanly separated.



IMPLEMENTATION & DEPLOYMENT

4.4 Scheduling & Placement Policies

Policy Name	Server-0 Permitted	Server-0 Blocked	Worker-0/1/2 Must Schedule Here	Topology Spread MaxSkew	Resource Request and Limits	Node Exporter-No Constraints
Description	Allows monitoring stacks to target server-0	Workloads lack toleration for NoSchedule taint	Mandates scheduling on worker nodes	Balances replicas across worker nodes	Declares explicit CPU and memory requests	Deployed without specific node constraints
Status	100%-Fully operational	0%-No workloads can be scheduled	100%-Fully operational	100%-Fully operational	Enforced-All resource requests and limits are actively monitored	All Nodes-Metrics are collected from every node

5. Experimental Evaluation

EXPERIMENTAL EVALUATION

5.1 Evaluation Objectives

- Goal: validate that a predictive MAS can initiate control actions before resource exhaustion — not merely implement another autoscaler
- Two experimental scenarios: (1) forecast-driven scaling vs. reactive HPA baseline, under a controlled workload ramp; (2) DCA supervisory resilience under injected agent failures

1

Proactive Scaling

Verify RSA actions trigger during WARNING pressure — not after CRITICAL saturation

2

Forecast-Driven Orchestration

Confirm telemetry → forecast → scaling decision pipeline operates end-to-end (MRA→PFA→RSA)

3

MAS vs Reactive Baseline

Determine measurable, qualitatively different resource-management behavior vs Kubernetes HPA

4

Supervisory Resilience

Validate DCA failure detection, autonomous recovery, and escalation (heartbeat / dual-layer confirmation)

5

Operational Feasibility

Confirm logical correctness and performance under real k3d scheduling, startup, and contention

EXPERIMENTAL EVALUATION

5.2 The Reactive Baseline: Kubernetes HPA

- HPA managing the stress-ng deployment — the standard, most representative reactive autoscaler in Kubernetes
- Configured with matching bounds to the MAS: min = 1, max = 5 replicas
- Scaling driven purely by CPU utilization, target 70% (aligned with the MAS CPU_WARNING threshold)
- Reacts only to current usage — no historical data, no forecasting, no projection beyond the current measurement
- Alternative baselines (event-driven autoscalers, custom Prometheus scalers) excluded — less representative & harder to isolate predictive contribution

HPA Proportional Control Formula

$$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \frac{\text{desiredMetricValue}}{\text{currentMetricValue}} \right\rceil$$

Scales only after utilization has already crossed the configured threshold.

Core Distinction

HPA — reacts to resource saturation in the present

MAS — anticipates future saturation via PFA forecasting (TimesFM 2.5)

EXPERIMENTAL EVALUATION

5.3 Metrics Definition

Metric	Definition	Purpose
M1 — Proactive Lead Time	T(HPA threshold crossed) − T(MAS proactive patch applied)	Principal metric: quantifies anticipation advantage
M2 — Scaling Response Latency	T(new replica ready) − T(load step applied)	End-to-end delay to additional capacity
M3 — CRITICAL Pressure Duration	Sum of intervals where utilization ≥ 85% of CPU limit	Overload-prevention capability
M4 — WARNING-to-Scale Delay	T(scale action) − T(first WARNING)	Responsiveness once trigger condition met
M5 — Replica-Seconds Overhead	Σ ($ReplicaCount_t \times \Delta t$)	Provisioned-capacity cost / efficiency
M6 — Forecast RMSE	RMSE between observed and TimesFM forecast values	Forecast accuracy (MAS only)

EXPERIMENTAL EVALUATION

5.4 Results & Comparative Analysis

Metric	HPA Baseline	MAS	Interpretation
M1 — Proactive Lead Time	0 s	+281 s	MAS scaled before workload escalation and before reactive threshold crossing
M2 — Response Latency	18 s	44 s	Extra latency = cost of forecasting/deliberation, offset by M1's lead time
M3 — CRITICAL Duration	0 s	0 s	Neither system saturated during the organic escalation phase
M4 — WARNING-to-Scale Delay	N/A	26 s	MAS converts WARNING pressure into a preventive action
M5 — Replica-Seconds Overhead	4530	3510	MAS used ~22.5% fewer replica-seconds than HPA
M6 — Forecast RMSE	N/A	291.84 mCPU	Forecasts accurate enough to support correct breach predictions

Headline result: the MAS initiated proactive scaling 281 s before the reactive baseline's trigger condition, while consuming ~22.5% fewer replica-seconds — forecast accuracy was sufficient to correctly anticipate breach conditions.

EXPERIMENTAL EVALUATION

5.5 Results & Comparative Analysis

Mode B

MRA FAILURE

Dual-layer confirmation (L1 heartbeat + L2 peer), autonomous restart, escalation after one failed recovery

Mode B*

RSA FAILURE

Immediate restart and simultaneous operator notification — the RSA is the sole scaling authority

Mode C

UNRESOLVABLE CONDITIONS

Immediate escalation for CAPACITY_OVERFLOW, EMERGENCY_INSUFFICIENT, CONFLICTING_AUTOSCALER

Mode B

PFA Recovery (Organic Failure)

RSA peer-monitoring detected PFA inactivity. L2-based confirmation, autonomous restart, timeout escalation, eventual service restoration without manual intervention.

Hot-path independence confirmed: MRA belief updates continued at 30-second intervals across all scenarios, including during RSA failure recovery. DCA coordination activity produced **no interruption** to the MRA→PFA→RSA operational pipeline. Architectural isolation between coordination and runtime operations confirmed.

6. Conclusion

CONCLUSION

6.1 Synthesis

Central objective validated: *predictive, distributed supervision is a feasible path from reactive to proactive Kubernetes resource management.*

+281 s	–22.5%	291.84	A · B · B* · C
proactive lead time	replica-seconds	mCPU forecast RMSE	resilience modes as specified

Five agents (MRA, PFA, RSA, DCA, SA) integrated forecasting, monitoring, messaging, persistence and supervision into one working prototype — with the operational pipeline preserved under failure.

CONCLUSION

6.2 Limitations

Forecasting cost

TimesFM 2.5 is large (~200M parameters; ~2 GiB requested / 3 GiB limit per PFA), raising deployment cost. With one PFA per workload, overhead scales roughly linearly with the number of services.

Controlled scope

Evaluated in a single-host k3d environment under a specific workload profile.

Open questions

Large-scale deployment and heterogeneous production workloads remain to be established.

CONCLUSION

6.3 Future Perspectives

Multivariate forecasting

Exploit inter-metric correlations for more precise trajectories and less conservative sizing.

Per-deployment fine-tuning

Use the accuracy records written to the Knowledge Base to adapt the model to each workload over time.

Extended temporal context

Multi-resolution observational windows to capture hourly, daily and weekly periodicity.

Agentic DCA

Autonomous resolution and cross-namespace cooperation between peer DCAs.

THANK YOU

Questions are welcome.